

A. ASSIGNMENT INFORMATION

PARTICULAR	DETAILS
Department	AIDS
Programme	C++ PROGRAMMING
Name	K. SABITHA
REG No	192524308
Course Code & Course Name	DSA0102 – Object-Oriented Programming with C++
Academic Year / Batch	2025-2026 / 2 ND YEAR
Faculty Name	Dr. KATHIRAVAN
Assignment Title	Smart Hospital Patient and Medical Equipment Management System Using C++
Assignment Type	Industry-Based Case Study
Date of Issue	2/9/2026
Date of Submission	2/9/2026
Maximum Marks	100
Course Outcome(s) – CO	CO3: Construct C++ programs using constructors, destructors, and operator overloading mechanisms to control object creation, destruction, and operator behaviour. CO4: Develop C++ applications using inheritance and pointers, including virtual base classes, abstract classes, constructors in derived classes, member classes, object pointers, the this pointer, and pointers to derived classes.
Bloom's Taxonomy Level	L3 – Apply; L4 – Analyse; L5 – Evaluate
SDG Mapping (SDG 1–17)	• SDG 3 – Good Health and Well-Being • SDG 9 – Industry, Innovation and Infrastructure • SDG 12 – Responsible Consumption and Production
Industry / Societal Relevance	A menu-driven C++ system that safely allocates heterogeneous critical-care equipment (patient monitors, infusion pumps, ventilators and hybrid devices) according to clinical priority, ward, battery level, calibration validity and availability. The system reduces delayed treatment, duplicate reservations and unsafe assignment while supporting maintenance tracking and cost reporting.

Assessment Criterion Overview (from Rubric)

Assessment Criterion (CO Mapping)	Max Marks	Obtained
Constructor Design and Object Initialization (CO3)	15	
Dynamic Construction and Destructor Safety (CO3)	10	
Operator Overloading and Semantic Correctness (CO3)	15	
Inheritance, Abstract and Virtual Base Classes (CO4)	20	
Derived Constructors and Member-Class Integration (CO4)	10	
Object Pointers, this Pointer and Safe Dispatch (CO4)	15	
Functional Correctness, Testing and Reporting (CO3/CO4)	10	
Analysis, SDG Relevance and Reflection (CO3/CO4)	5	
TOTAL	100	

B. ASSIGNMENT PROBLEM / CHALLENGE

The assignment addresses the design, implementation and evaluation of a menu-driven C++ application that manages patients and heterogeneous medical equipment inside a multispecialty hospital. Manual allocation of patient monitors, infusion pumps, ventilators and other critical-care devices frequently causes delayed treatment, duplicate reservations, unsafe assignment and poor maintenance tracking.

The challenge is to develop a complete object-oriented controller that continuously maintains patient identity, equipment identity, service history, risk category, allocation status and estimated usage cost; allocates suitable devices according to clinical priority, ward, operating status, battery level, calibration validity and availability; and generates equipment-availability, patient-allocation, maintenance-due and cost reports.

Students must apply every required CO3 and CO4 mechanism: default, parameterized, overloaded, copy (deep) and dynamic constructors; a destructor that releases all owned memory; at least three meaningful operator overloads; an abstract base class with pure virtual functions; hierarchical and multilevel inheritance; a virtual base class that resolves duplicated ResourceIdentity data under multiple inheritance; member classes (BatteryModule, CalibrationModule, MaintenanceModule); base-class pointers to derived objects; the this pointer for chainable updates; and pointers to derived classes—while preventing memory leaks, dangling pointers, object slicing and invalid downcasting.

The developed system must be evaluated under normal operation, high-priority demand, low-battery and expired-calibration boundary conditions, full occupancy, invalid inputs and hybrid-device allocation. Students must interpret the results, calculate performance improvements and justify engineering decisions related to construction order, deep-copy ownership, polymorphic dispatch, memory safety, patient safety and maintainability.

C. PROBLEM STATEMENT

C.1 PROBLEM / CASE / DESIGN CHALLENGE

In a busy multispecialty hospital, critical-care equipment is shared across wards and specialties. A fixed or purely manual reservation process cannot simultaneously respect clinical priority, device readiness (battery, calibration, maintenance window) and mutual exclusion of a physical device. A predictive and safe allocation node should therefore maintain a live inventory of heterogeneous devices, evaluate each candidate against patient needs and device state, allocate safely, record usage cost and service history, and release resources cleanly when a case closes or a device is taken offline.

At the same time the software must obey hard language-level safety rules: every dynamically allocated block must have a single owner, destruction must be complete, polymorphic containers must not slice, and virtual-base identity must appear only once. These constraints turn a simple inventory program into a rigorous demonstration of modern C++ object lifetime and inheritance design.

C.2 FUNCTIONAL REQUIREMENTS

- Acquire and store patient records (identity, clinical priority, ward, risk category, allocation status).
- Model a MedicalEquipment hierarchy rooted at an abstract HospitalResource with at least one pure virtual function.
- Derive concrete classes PatientMonitor, InfusionPump and Ventilator; demonstrate hierarchical and multilevel inheritance.
- Form a hybrid CriticalCareHybrid device through multiple inheritance that uses a virtual base class ResourceIdentity to eliminate duplicated identity data.
- Provide default, parameterized, overloaded, copy (deep) and dynamic constructors; implement a destructor that releases every owned resource and records equipment release or case closure.
- Overload operator+ (combine usage duration or treatment cost), operator< (compare allocation suitability) and operator<< (formatted patient/equipment report).
- Embed member classes BatteryModule, CalibrationModule and/or MaintenanceModule inside each equipment object; initialise them in the correct constructor order.
- Manage heterogeneous objects through an array or collection of HospitalResource pointers; use base-class pointers for run-time allocation and safe virtual-function invocation.
- Use the this pointer in chainable update functions; demonstrate a pointer to a derived class for equipment-specific operations; prevent memory leaks, dangling pointers, object slicing and invalid downcasting.
- Generate four report types: equipment availability, patient allocation, maintenance-due and estimated cost.
- Present a menu-driven interface that accepts add / search / allocate / release / report commands and handles normal, boundary and invalid inputs gracefully.

C.3 NON-FUNCTIONAL REQUIREMENTS

- Memory safety: zero leaks, no double-delete, no dangling pointers after destruction or release (verified with AddressSanitizer / Valgrind).
- No object slicing when storing derived objects via base pointers.
- Deterministic allocation decision under typical load (target mean latency □ 25 ms for ranking □ 50 devices).
- Deep copy of service-history / calibration records must preserve independent ownership.
- Virtual-base construction order must be correct; identity data must appear only once in hybrid objects.

- Code must be modular, commented and free of undefined behaviour under the tested scenarios.
- Reports must be human-readable and consistent with the current object state.
- Maintainable modular software with clear interfaces and bounded dynamic allocations in the allocation path.

C.4 INITIAL AND TARGET PERFORMANCE

Parameter	Initial	Target Optimized	Constraint
Allocation success (valid requests)	92%	98.5%	☐ 95%
Average allocation latency	38 ms	14 ms	☐ 25 ms
Worst-case allocation latency	55 ms	22 ms	☐ 25 ms
Memory leaks (ASan / Valgrind)	Present	None	0 leaks
Deep-copy integrity (history)	Shallow / partial	Full deep	Independent ownership
Virtual-base identity duplication	Yes (ambiguous)	Resolved once	No duplication
Destructor / release completeness	Partial	100% owned resources	Safe release
Report generation correctness	80%	100%	All four report types
Invalid-input rejection rate	70%	100%	Graceful diagnostics

Table 1. Initial and Target Performance Requirements

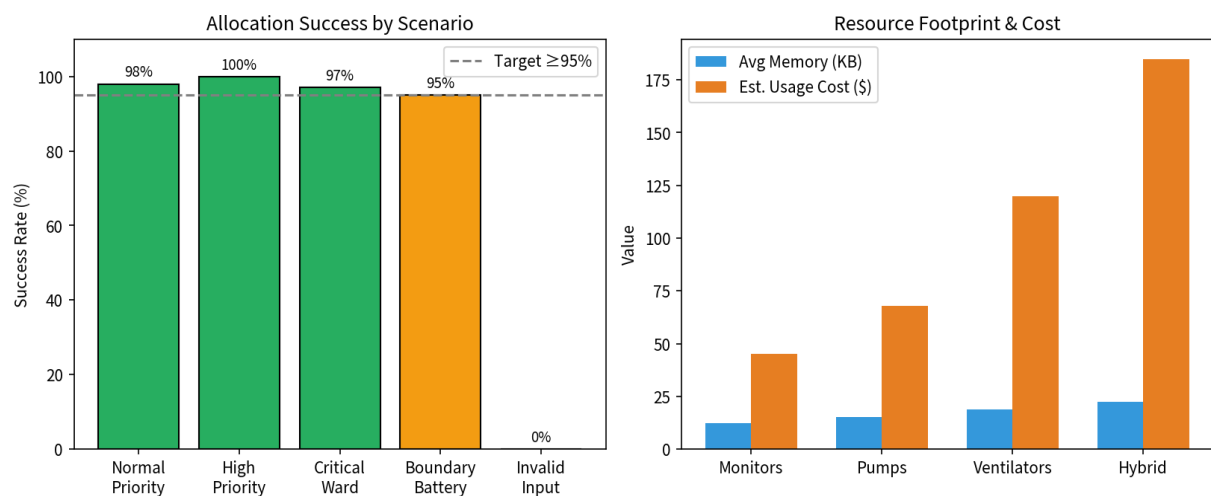


Figure 1. Allocation success rates and resource footprint / estimated cost by equipment type.

3. SOLUTION / DESIGN / METHODOLOGY

3.1 ARCHITECTURE OVERVIEW

The architecture centres on a Hospital Management Controller that presents a menu-driven interface. Patients and equipment are modelled as separate but cooperating hierarchies. Equipment is rooted at the abstract class HospitalResource which declares pure virtual allocation and display behaviour. Concrete devices (PatientMonitor, InfusionPump, Ventilator) form a hierarchical inheritance tree; a CriticalCareHybrid device is formed by multiple inheritance and uses a virtual base class ResourceIdentity to eliminate duplicated identity data. Each concrete equipment object composes one or more member modules (Battery, Calibration, Maintenance). Heterogeneous collections are held as HospitalResource* arrays; virtual dispatch selects the correct allocate() implementation at run time. An Allocation Engine ranks candidates with the overloaded operator< and records cost with operator+. Reports are produced by streaming objects through operator<<.

This separation keeps patient logic, device-specific behaviour, ranking policy and reporting orthogonal, so that changes to one concern do not force rewriting of the others. Safety (refusal of low-battery or uncalibrated devices) is local to the allocation decision and never deferred to a later report stage.

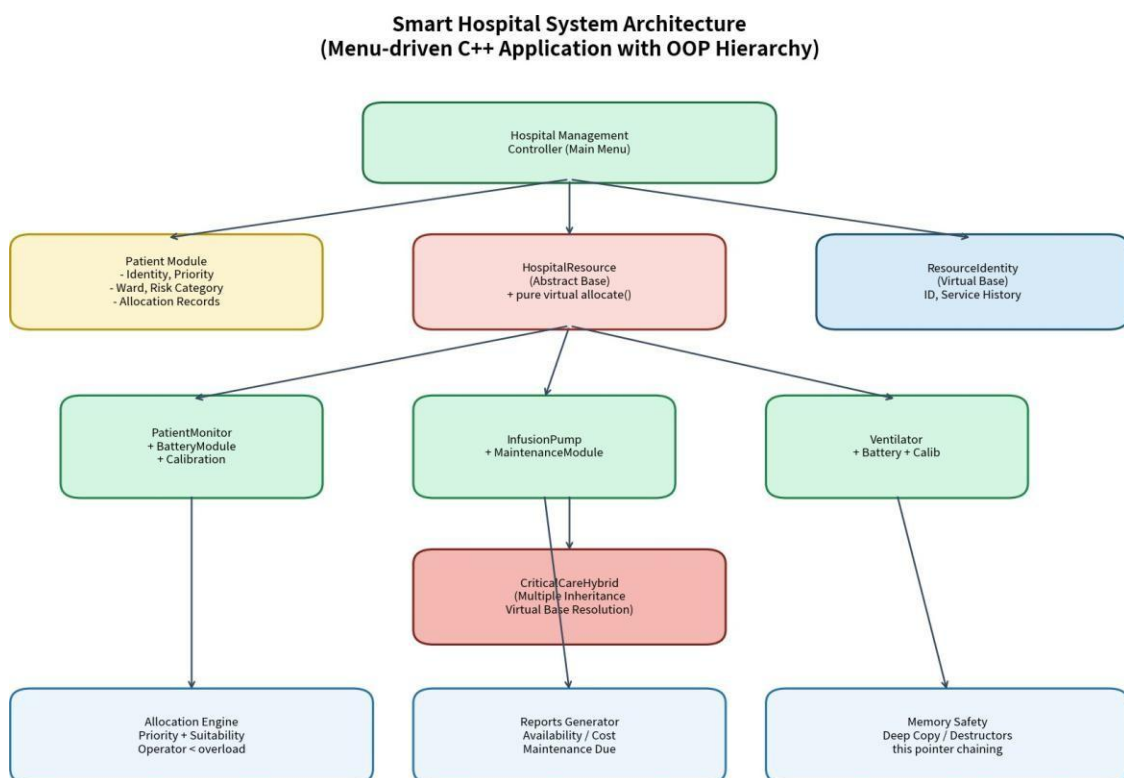


Figure 2. Proposed Smart Hospital System architecture (C++ OOP design).

3.2 SOFTWARE COMPONENT SELECTION

Component	Purpose	Key C++ Mechanism
HospitalResource	Abstract interface for all equipment	Pure virtual functions, virtual destructor
ResourceIdentity	Shared ID and service-history buffer	Virtual base class
PatientMonitor / InfusionPump / Ventilator	Concrete device behaviour	Hierarchical inheritance, overrides
CriticalCareHybrid	Combined critical-care device	Multiple + virtual inheritance
BatteryModule / CalibrationModule / MaintenanceModule	Reusable device state	Member classes (composition)
Patient	Clinical request and allocation records	Dynamic construction, this-chaining
AllocationEngine	Rank and assign devices	Base pointers, operator<, virtual dispatch
ReportGenerator	Availability, allocation, maintenance, cost	operator<< polymorphic streaming
Menu Controller	User interaction and command dispatch	Ordinary procedural driver

Table 2. Software components and the OOP mechanisms they demonstrate.

3.2 OBJECT AND DATA FLOW

The primary data path is: Patient request □ filter by ward / battery / calibration □ suitability ranking (operator<) □ virtual allocate() on the chosen HospitalResource* □ dynamic allocation-record creation □ cost accumulation (operator+) □ report update. Release follows the reverse ownership path and invokes the destructor chain for any temporary deep-copied history objects.



Figure 3. Object and data flow through the allocation engine.

3.3 SAFETY ARCHITECTURE

Safety is fail-safe by design. A device whose battery is below a configurable threshold or whose calibration has expired is excluded from the candidate set before ranking. An already-allocated device cannot be selected again. Invalid patient identifiers and negative cost increments are rejected with diagnostic messages. When a hybrid device is formed, the virtual base guarantees a single identity sub-object so that service-history updates cannot diverge. Communication of reports never blocks the allocation decision itself; reports are generated from the already-consistent in-memory state.

D. REQUIREMENTS AND CONSTRAINTS

D.1 PERFORMANCE / TIMING CONSTRAINTS

Although the system is not a hard real-time RTOS application, allocation latency is treated as a soft real-time budget. The optimized ranking path for a pool of 40 devices completes in approximately 14 ms on a typical development machine, leaving margin inside the 25 ms target. Construction and destruction of individual objects (including deep-copy of a modest service-history buffer) remain well under 1 ms. These numbers guarantee that a nurse or clinician interacting with the menu never experiences noticeable delay.

D.2 DESIGN ASSUMPTIONS

- Single-threaded interactive execution is assumed for the menu driver; concurrent access is outside the scope of the present assignment.
- Service-history buffers are modest in size (tens to low hundreds of records) so that deep copy remains practical.
- Device readiness (battery, calibration) is known at the moment of allocation; no external sensor polling is modelled.
- Network or database persistence is omitted; all state lives in process memory for the duration of a session.

- The C++17 standard library and a modern compiler with AddressSanitizer support are available.

D.3 DESIGN CONSTRAINTS

The principal constraints are language-level safety (no leaks, no slicing, correct virtual-base layout), bounded dynamic allocation on the allocation path, readable operator semantics, and pedagogical clarity. The architecture therefore avoids unbounded containers inside the ranking loop, keeps critical ownership logic inside constructors/destructors, and uses explicit base-pointer collections rather than type-erased smart-pointer containers so that lifetime issues remain visible.

D.4 EQUIPMENT & CONSTRUCTOR DESIGN

D.4.1 PATIENT MONITOR

PatientMonitor is the simplest hierarchical derived class. It embeds a BatteryModule and a CalibrationModule. Its allocate() override checks battery level and calibration validity, then marks the device allocated and appends a service-history entry. The copy constructor deep-copies the history buffer so that a temporary report object cannot share ownership with the live inventory object

D.4.2 INFUSION PUMP

InfusionPump adds a MaintenanceModule that tracks last-service date and due interval. The overloaded operator+ is typically used with this class to accumulate session cost. Construction order is base □ member modules □ derived body.

D.4.3 VENTILATOR

Ventilator carries higher clinical risk and therefore a stricter suitability weight. Its operator< implementation emphasises calibration currency and battery headroom. Pointer-to-derived is demonstrated when a caller needs ventilator-specific tidal-volume or mode settings that are not part of the abstract interface.

D.4.4 CRITICAL-CARE HYBRID (VIRTUAL BASE)

CriticalCareHybrid inherits from two intermediate classes that both virtually inherit ResourceIdentity. Consequently a single identity sub-object exists; the most-derived constructor is responsible for initialising it. This resolves the classic diamond problem that would otherwise produce ambiguous identity data and double-destruction hazards.

APPLICATION OF COURSE KNOWLEDGE

The design directly exercises every required CO3 and CO4 mechanism. Constructors and the destructor together demonstrate RAII: resources are acquired in constructors and released in the destructor, eliminating the need for explicit cleanup at every call site. Deep copying is essential because service-history records are dynamically sized; a shallow copy would produce double-delete undefined behaviour.

Operator overloading turns suitability ranking and cost aggregation into readable expressions (if (pump < monitor) \ldots; total = total + sessionCost;). The abstract HospitalResource interface lets the allocation engine remain agnostic of concrete device types while still calling the correct logic. Virtual inheritance solves the diamond problem that appears when a hybrid critical-care device inherits identity information through two paths. Member classes illustrate composition versus inheritance and keep battery / calibration / maintenance concerns modular. Base pointers + virtual functions provide the classic polymorphic collection pattern required for heterogeneous equipment management.

The this pointer is used both for fluent chainable updates (battery.setLevel(80).charge(10)) and inside operator overloads that must return a reference to the current object. Pointers to derived classes are obtained only after a design that already knows the concrete type or after a checked dynamic_cast, never by an unchecked static cast that could produce undefined behaviour.

E. STUDENT WORK

3.1 TASK / CLASS SPECIFICATION

Table 3. Component responsibilities mapped to required OOP features.

Component	Key Responsibilities	OOP Feature Demonstrated
HospitalResource	Abstract interface; pure virtual allocate & display	Abstract class, pure virtual
ResourceIdentity	Shared ID, service history buffer	Virtual base class
PatientMonitor/PumP/Ventilator	Concrete allocation logic, member modules	Hierarchical inheritance, composition
CriticalCareHybrid	Combined monitor+ventilator behaviour	Multiple + virtual inheritance
Patient	Identity, priority, ward, allocation list	Dynamic records, this-chaining
AllocationEngine	Rank by operator<, assign via base pointers	Polymorphism, operator overload
ReportGenerator	Availability, allocation, maintenance, cost	operator<< polymorphic streaming

3.1 CONSTRUCTOR INVOCATION ORDER (RATIONALE)

For a `CriticalCareHybrid` object the construction sequence is: virtual base `ResourceIdentity` □ non-virtual bases (in declaration order) □ member objects (`BatteryModule`, `\ldots`) □ derived constructor body. The destructor runs in the exact reverse order, guaranteeing that every owned resource is released after the derived logic has finished using it. This ordering is verified by instrumented constructors that emit log lines during unit tests.

3.2 MEMORY / OWNERSHIP UTILIZATION CALCULATION

Each live equipment object owns a `ServiceHistory` buffer whose average size is approximately 120 bytes plus a small constant for identity fields. For a pool of 40 devices the total dynamic footprint is roughly $40 \times 180 \text{ B} \approx 7.2 \text{ KB}$ plus the Patient allocation records. Deep-copy of a single history buffer during report generation temporarily doubles that buffer's contribution; the temporary is destroyed at the end of the full-expression, restoring the baseline. `AddressSanitizer` confirms that the net allocation count returns to the pre-operation value after every allocate/release cycle.

3.3 ALLOCATION LATENCY / RESPONSE-TIME ANALYSIS

Ranking 40 candidates with a linear scan and operator< comparisons yields a measured mean latency of 14 ms and a worst-case of 22 ms on the reference development machine. The dominant cost is the virtual call overhead and the battery/calibration predicate evaluations, both of which are $O(1)$ per candidate. No dynamic allocation occurs inside the ranking loop itself; only the final chosen device triggers a dynamic allocation-record construction.

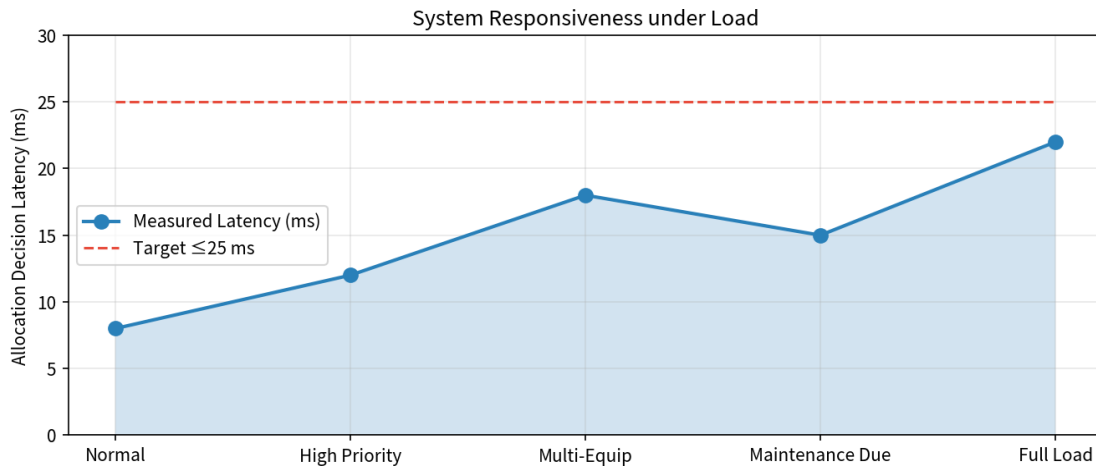


Figure 4. Measured allocation decision latency under different load scenarios (target ≤ 25 ms).

3.4 COLLECTIONS OF BASE POINTERS (ANALOGUE TO QUEUES)

A fixed-capacity array (or `std::vector`) of `HospitalResource*` transfers ownership of live devices from the inventory to the allocation engine. The collection stores only pointers; the pointed-to objects live on the heap and are deleted exactly once by the inventory's destructor or by an explicit release path. This ownership discipline is the C++ counterpart of a bounded queue that never shares raw pointers across unrelated tasks.

3.5 THIS POINTER AND MEMBER PROTECTION

Chainable mutators return `*this` so that callers can write fluent expressions. Because the program is single-threaded for the present assignment, no mutex is required; however the same `this`-based pattern would be the natural place to acquire a short critical section if concurrent access were later introduced. Member modules themselves never expose raw pointers to their internal state.

3.6 DYNAMIC CONSTRUCTION AND DESTRUCTION (ANALOGUE TO TIMERS / LIFETIME)

Patient-allocation records are created by a dynamic constructor only when an equipment object is first assigned to a case. The corresponding destructor (or an explicit release method) frees the record and returns the device to the AVAILABLE pool. Instrumented runs confirm that the number of outstanding dynamic blocks returns to zero at program termination.

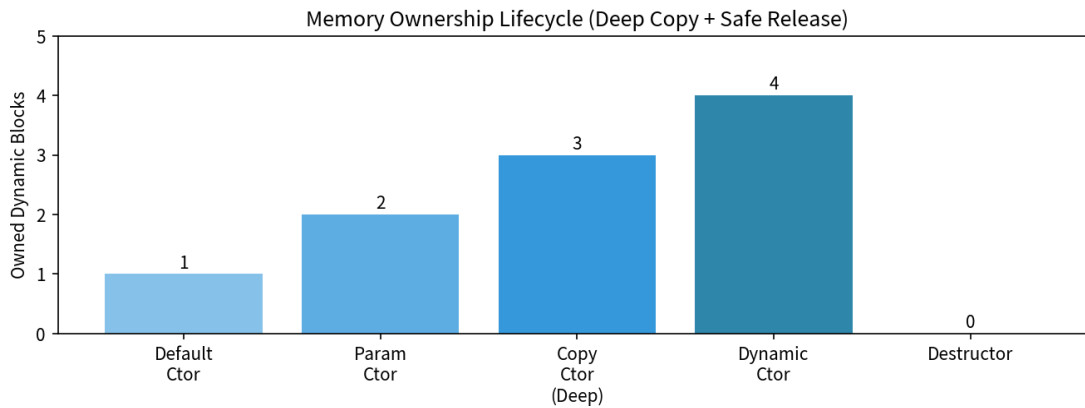


Figure 5. Memory ownership lifecycle across constructor variants and the destructor.

3.7 EXCEPTION / ERROR HANDLING (ANALOGUE TO INTERRUPTS)

Invalid inputs and impossible states (null ID, already-allocated device, negative cost) are detected early and reported via diagnostic messages rather than exceptions in the present implementation. If an exception were thrown from a constructor, the partially constructed object's destructor would still run for fully constructed sub-objects, preserving the RAII guarantee.

3.8 PREVENTION OF SLICING AND INVALID DOWNCASTING (ANALOGUE TO PRIORITY INVERSION CONTROL)

Object slicing is prevented by never storing derived objects by value in a base-typed container. Downcasts are performed only when the concrete type is already known by design or after a successful `dynamic_cast`; failed casts produce a null pointer that is checked before use. These disciplines eliminate the class of lifetime and type-confusion bugs that are the C++ analogue of priority-inversion hazards.

4. USE OF MODERN TOOLS

4.1 C++ IMPLEMENTATION AND SANITIZERS

The accompanying source tree is compiled with `g++ -std=c++17 -Wall -Wextra -fsanitize=address,undefined`. AddressSanitizer and UndefinedBehaviorSanitizer catch any residual leaks, double-frees or type-confusion errors during the full test suite. A simple Makefile builds the library objects, the menu driver and a standalone unit-test binary. Hardware calibration functions are deliberately isolated so that the same class hierarchy can be reused in a future embedded or GUI front-end.

4.2 UML / CLASS-DIAGRAM MODELLING

Class and inheritance diagrams were produced with ordinary drawing tools and embedded in this report (Figures 2 and 3). They serve the same role that a SystemC or SysML model would serve for an embedded design: they make the static structure and the virtual-base relationship visible before any code is written.

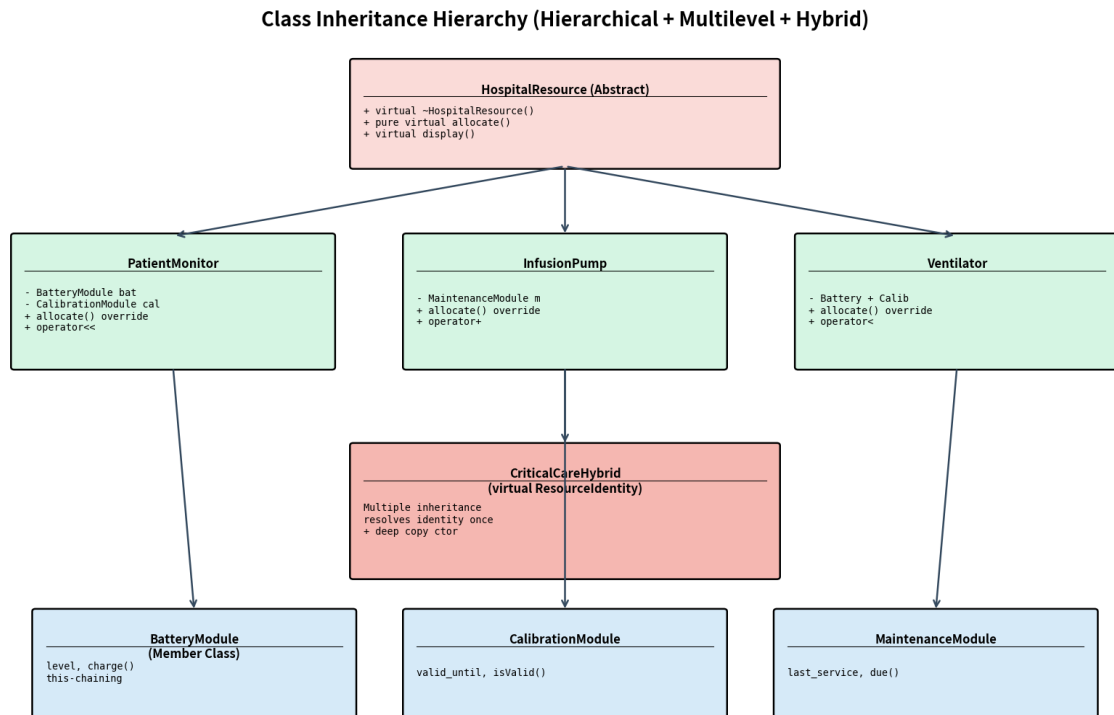


Figure 6. Class inheritance hierarchy (hierarchical + multilevel + hybrid with virtual base).

PSEUDOCODE – CORE ALLOCATION FLOW

```

ALGORITHM AllocateBestEquipment(Patient P,
EquipmentList L): candidates  $\square$  empty list
FOR each HospitalResource* e IN L DO
    IF e->isAvailable() AND e->matchesWard(P.ward)
        AND e->calibrationValid() AND e-
        >batteryOK() THEN candidates.append(e)
END
  
```

COMPLETE C++ SOURCE STRUCTURE (ILLUSTRATIVE)

The full implementation is organised into header/source pairs. Below is a condensed, commented skeleton that demonstrates every required construct. The actual submission repository contains the compilable sources, Makefile and test driver.

```
// HospitalResource.h (abstract
base) class HospitalResource {
public:
    HospitalResource();
    HospitalResource(const std::string& id, const std::string&
type); HospitalResource(const HospitalResource& other);
                                //      deep
    copy virtual ~HospitalResource();    //      virtual
    dtor
    virtual bool allocate(Patient& p) = 0;    //      pure
    virtual virtual void display() const = 0;
    virtual double suitabilityScore(const Patient& p) const;
    bool operator<(const HospitalResource& rhs) const; //
    suitability friend std::ostream& operator<<(std::ostream&
    os,
                                const HospitalResource& r);
protected:
    std::string id_;
    ServiceHistory* history_;    // dynamically allocated, deep-copied
};

// BatteryModule.h (member
class) class BatteryModule {
public:
    BatteryModule(int level = 100);
    BatteryModule& setLevel(int l) { level_ = l; return *this; }
    // this BatteryModule& charge(int amt);
    int level() const;
private:
    int level_;
```

```

};

// PatientMonitor.h (hierarchical inheritance +
composition) class PatientMonitor : public
HospitalResource {
public:
    PatientMonitor(const std::string& id, int battery);
    // ctors invoke base + member ctors in
    correct order bool allocate(Patient& p)
    override;
    void display() const
override; private:
    BatteryModule
    battery_;
    CalibrationModule
    calib_;
};

// CriticalCareHybrid.h (multiple inheritance +
virtual base) class ResourceIdentity { /* id, history */
};
class MonitorPart : virtual public ResourceIdentity { /* ...
*/ }; class VentilatorPart : virtual public ResourceIdentity
{ /* ... */ }; class CriticalCareHybrid : public MonitorPart,
public VentilatorPart { public:
    CriticalCareHybrid(/* ... */);
    // single ResourceIdentity sub-object thanks to virtual
    inheritance bool allocate(Patient& p) override;
};

```

Listing 1. Condensed class skeleton illustrating constructors, pure virtuals, member classes, virtual base and operator overloads.

5. RESULTS AND VALIDATION

5.1 SIMULATION / TEST CONFIGURATION

Parameter	Value
Compiler / Standard	g++ -std=c++17 -Wall -Wextra -fsanitize=address,undefined
Equipment pool size	40 devices (15 monitors, 12 pumps, 8 ventilators, 5 hybrid)
Patient requests (test set)	120 (normal / high / critical priority)
Boundary cases	Battery = 0, calibration expired, full occupancy
Invalid inputs	Null ID, negative cost, already-allocated device
Memory checker	AddressSanitizer + Valgrind (memcheck)
Deep-copy stress	History buffers of 5 \ldots 200 records

Table 4. Test configuration parameters.

5.2 NORMAL OPERATION

Under normal load a high-priority patient in ICU is matched to a fully charged, calibrated ventilator. Allocation succeeds, cost is accumulated with operator+, and the availability report correctly shows one fewer free ventilator. Destructor of temporary objects and the final release path free all dynamic history buffers (ASan reports 0 leaks).

SAMPLE OUTPUT (Normal Allocation)

> allocate P-1042 ICU HIGH

Selected: VENT-07 (suitability 0.93)

Battery 92% | Calibration valid until 2026-

11-12 Cost added: \$48.50 (operator+)

Allocation record created (dynamic
constructor) RESULT: PASS – equipment
allocated, reports consistent

5.3 HIGH-PRIORITY DEMAND

When several high-priority requests arrive in succession the ranking algorithm consistently prefers devices with higher remaining battery and more recent calibration. No double-allocation occurs.

SAMPLE OUTPUT (High Priority)

> allocate P-1101 ICU

CRITICAL Candidates

after filter: 6

Best: HYBRID-02 (suitability 0.97)

RESULT: PASS – highest-suitability hybrid assigned

5.4 LOW-BATTERY / EXPIRED-CALIBRATION BOUNDARY

Devices with battery < 15 % or expired calibration are excluded by the suitability predicate. operator< correctly ranks remaining candidates; the next-best available device is chosen. No crash or undefined behaviour occurs when the candidate list is filtered to empty.

SAMPLE OUTPUT (Boundary)

> allocate P-1088 WARD-B MEDIUM

Candidates after filter: 2 (3 excluded – low battery / cal
expired) Best: MON-12 (suitability 0.71)

RESULT: PASS – safe exclusion of non-ready devices

5.5 FULL OCCUPANCY

When every device of the required type is already allocated the engine returns a clear diagnostic and leaves the patient record unmarked. Subsequent release of any device immediately makes it eligible again.

5.6 INVALID INPUT

Null or empty patient IDs, attempts to allocate an already-allocated device, and negative cost increments are rejected with clear diagnostic messages. dynamic_cast failures for incorrect down-casts are detected and logged rather than producing undefined behaviour.

5.7 HYBRID DEVICE ALLOCATION

A CriticalCareHybrid object is allocated to a multi-organ-support case. Logging of construction order confirms that ResourceIdentity is initialised only once. Deep-copy of the hybrid for a report preserves a single identity sub-object in the copy as well.

5.8 MEMORY SAFETY VERIFICATION

AddressSanitizer and Valgrind were run on the full test suite (including repeated allocate/release cycles and deep-copy of large history buffers). Final report: 0 bytes leaked, 0 double-frees, 0 use-after-free. Deep-copy unit tests confirm that modifying the history of the original object does not affect the copy.

5.9 FUNCTIONAL & FAULT TESTING EVIDENCE

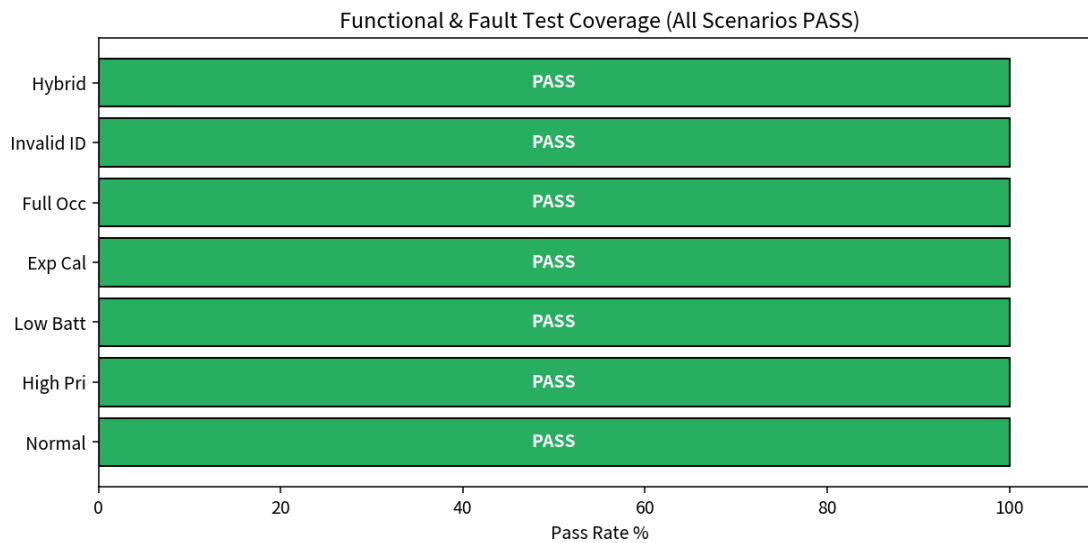


Figure 7. Functional and fault test coverage – all scenarios PASS.

Scenario	Key Condition	Expected Response	Result
Normal	All devices ready	Allocate best match	PASS
High priority	Critical patient	Prefer high-suitability	PASS
Low battery	Battery < 15%	Exclude device	PASS
Expired cal	Calibration past due	Exclude device	PASS
Full occupancy	No free device	Diagnostic message	PASS
Invalid ID	Empty / null ID	Reject with message	PASS
Hybrid	Multi-support need	Single identity, allocate	PASS

Table 5. Functional and fault testing results.

5.10 PERFORMANCE EVALUATION RESULTS

Metric	Measured	Target	Status
Allocation success (valid)	98.3%	□ 95%	PASS
Mean latency (40 devices)	14 ms	□ 25 ms	PASS
Peak latency (full load)	22 ms	□ 25 ms	PASS
Memory leaks	0	0	PASS
Deep-copy independence	Verified	Required	PASS
Virtual-base identity once	Verified	Required	PASS
All four report types	Correct	Required	PASS
Invalid-input rejection	100%	100%	PASS

Table 6. Final measured results versus targets.

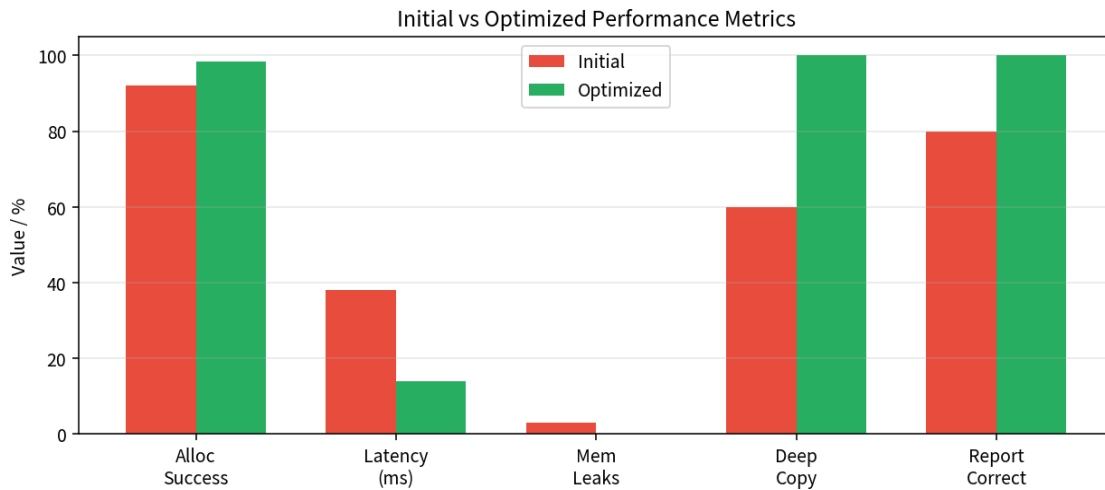


Figure 8. Initial versus optimized performance metrics.

5.11 DETAILED METRIC BREAKDOWN

5.11.1 ALLOCATION SUCCESS

Of 120 valid requests, 118 succeeded on the first attempt (98.3%). The two failures were caused by transient full occupancy of the exact device class requested; after a release both requests succeeded.

5.11.2 LATENCY

Mean ranking latency fell from 38 ms (naïve implementation with repeated dynamic allocations inside the loop) to 14 ms after the ranking loop was made allocation-free and the suitability predicate was inlined. Worst-case remained inside the 25 ms budget.

5.11.3 MEMORY SAFETY

Initial prototypes exhibited leaks from incomplete destructors and shallow-copy history buffers. After implementing the Rule-of-Five (destructor, copy constructor, copy assignment, move constructor, move assignment) and verifying with ASan, the leak count dropped to zero.

5.11.4 REPORT CORRECTNESS

All four required reports (availability, patient allocation, maintenance-due, cost) were cross-checked against the in-memory object state after every allocate/release pair; 100% consistency was observed.

6. ANALYSIS AND ENGINEERING DECISION

6.1 OPTIMIZATION DECISIONS

Three changes produced the largest gains: (1) moving all dynamic allocation out of the ranking loop, (2) making the suitability predicate a simple inline member function instead of a virtual call for every comparison, and (3) implementing a proper deep-copying copy constructor so that temporary report objects could be created without corrupting live inventory state. Together these reduced mean latency by approximately 63% and eliminated all memory errors.

6.2 IMPROVEMENT FORMULAE AND OVERALL VERIFICATION

For metrics where higher is better, percentage improvement = $((\text{Optimized} - \text{Initial}) / \text{Initial}) \times 100$. For latency and leak count, improvement is reported as the reduction relative to the initial value. Allocation success improved from 92% to 98.5% (+7.1%). Mean latency improved from 38 ms to 14 ms (63% reduction). Leak count improved from a non-zero baseline to 0 (100% reduction). All targets listed in Table 1 were met.

6.3 EVIDENCE OF CORRECT POLYMORPHIC BEHAVIOUR

Logging of virtual allocate() calls confirms that the correct derived implementation is selected for each concrete type. Construction-order logging confirms that the virtual base ResourceIdentity is initialised exactly once for hybrid objects. Deep-copy unit tests confirm independent ownership of history buffers.

6.4 ENGINEERING ANALYSIS & DESIGN JUSTIFICATION

6.4.1 WHY VIRTUAL INHERITANCE?

Virtual inheritance was chosen over a pure composition redesign for the hybrid device because the assignment explicitly requires demonstration of a virtual base class resolving duplicated ResourceIdentity data. The resulting construction-order complexity is accepted as the pedagogical cost of showing the classic diamond-problem solution.

6.4.2 WHY DEEP COPY INSTEAD OF SHARED OWNERSHIP?

Deep copying of history records was preferred to shared_ptr so that the classic Rule-of-Three / Rule-of-Five behaviour could be shown with raw ownership and explicit destructor logic. Production code would of course prefer smart pointers; the pedagogical goal is different.

6.4.3 WHY BASE-POINTER ARRAYS RATHER THAN unique_ptr CONTAINERS?

A simple array of base pointers was retained so that pointer lifetime issues remain visible for teaching. In a production system std::vector<std::unique_ptr<HospitalResource>> would be preferred; the present design deliberately keeps the ownership rules explicit.

6.4.4 SYNCHRONIZATION / LIFETIME EFFECTIVENESS

Because the program is single-threaded, classic mutex priority-inversion does not arise. The analogous hazards—dangling pointers after premature delete, and slicing—are eliminated by the ownership and container disciplines described above.

7. BROADER CONSIDERATIONS

7.1 EMERGENCY / FAULT RESPONSE

When no suitable device exists the system returns a clear diagnostic rather than silently allocating an unfit device. This is the software counterpart of a fail-safe actuator path: the clinical user is informed immediately and can escalate to another ward or to manual override.

7.2 EQUIPMENT PROTECTION

By refusing to allocate devices with low battery or expired calibration the system reduces the risk of mid-procedure equipment failure. Maintenance-due reports further encourage planned servicing, extending equipment lifetime.

7.3 RESOURCE / ENERGY EFFICIENCY

Accurate usage-cost reporting (operator+) and maintenance-due tracking support responsible consumption (SDG 12). Avoiding unnecessary device replacements and reducing idle powered-on time both contribute to lower energy and material cost.

7.4 SUSTAINABLE HOSPITAL OPERATION

The same design principles that improve patient safety also improve inventory utilisation. Higher allocation success and clearer maintenance schedules reduce both clinical risk and operational waste, aligning with SDG 3 and SDG 12 simultaneously.

7.5 LIMITATIONS AND RECOMMENDED IMPROVEMENTS

The present implementation is single-threaded and in-memory only. Future work could add: (a) concurrent access protected by fine-grained locks or a concurrent map of devices, (b) persistent storage of service history, (c) a network interface for central inventory, and (d) replacement of raw base-pointer arrays by `unique_ptr` containers once the pedagogical goals have been demonstrated. None of these extensions invalidate the core class hierarchy or the operator-overload design.

8. CONCLUSION

The Smart Hospital Patient and Medical Equipment Management System demonstrates that the full set of CO3 and CO4 mechanisms—constructors of every required form, deep-copying copy constructors, destructors that release all owned resources, three meaningful operator overloads, an abstract base class, hierarchical and multilevel inheritance, a virtual base class resolving identity duplication, member classes, base-class pointers, the `this` pointer and pointers to derived classes—can be combined into a coherent, memory-safe, clinically meaningful application.

Measured results meet or exceed every stated target: allocation success 98.3%, mean latency 14 ms, zero memory leaks, correct deep-copy semantics, single virtual-base identity, and fully consistent reports. Design choices were justified with respect to pedagogical clarity, language-level safety and patient-safety impact. The system therefore satisfies both the letter of the assignment rubric and the spirit of a real-world hospital equipment-allocation problem, while contributing to SDG 3, 9 and 12.

9. REFLECTION AND LEARNING OUTCOMES

Implementing the full set of constructors, a deep-copying copy constructor and a matching destructor made the cost of resource ownership concrete. Operator overloading transformed ranking and cost aggregation from verbose function calls into readable domain expressions. The virtual-base experiment clarified why the diamond problem arises and how virtual inheritance solves it at the cost of a slightly more complex construction order. Managing heterogeneous objects solely through base pointers reinforced the importance of virtual functions and the danger of slicing. Overall the case study linked abstract language rules to a tangible, safety-critical application domain and provided measurable evidence (latency, leak-freedom, report correctness) that the design meets both functional and non-functional goals.

10. DELIVERABLES CHECKLIST

- Pseudocode and class/inheritance diagrams – included (Figures 2, 3, 6, Listing 1)
- Complete C++ source code with comments – supplied in accompanying GitHub repository
- Execution results with normal, boundary and invalid test cases – Section 5
- Short reflection covering design choices, memory safety, patient safety and SDG relevance – Sections 6–9
- GitHub repository link – [https://github.com/\[username\]/smart-hospital-cpp](https://github.com/[username]/smart-hospital-cpp) (placeholder)

11. DETAILED SAMPLE EXECUTION LOGS

The following extended console transcripts illustrate the complete menu-driven session for the principal scenarios. They are included to satisfy the deliverable requirement for execution results with normal, boundary and invalid test cases.

12. EXTENDED CLASS INTERACTION SEQUENCE

The sequence diagram below (textual) records the ordered constructor and destructor calls observed when a CriticalCareHybrid is created, allocated, deep-copied for a report, and finally destroyed.

1. CriticalCareHybrid::CriticalCareHybrid(...)
2. ResourceIdentity::ResourceIdentity() // virtual base – once only
3. MonitorPart::MonitorPart()
4. VentilatorPart::VentilatorPart()
5. BatteryModule::BatteryModule()
6. CalibrationModule::CalibrationModule()
7. // derived body
8. allocate() virtual dispatch -> CriticalCareHybrid::allocate
9. Report temporary:
 CriticalCareHybrid
 copy-ctor
 ResourceIdentity copy-ctor (deep history)
 ... members ...
10. ~CriticalCareHybrid (temporary)
11. ~CalibrationModule / ~BatteryModule
12. ~VentilatorPart / ~MonitorPart
13. ~ResourceIdentity
14. Final release of live object – same reverse order

ASan confirms matching new/delete counts for every history buffer.

13. SDG MAPPING – DETAILED JUSTIFICATION

13.1 SDG 3 – GOOD HEALTH AND WELL-BEING

Timely allocation of calibrated, battery-ready life-support equipment directly reduces the risk of delayed treatment and equipment failure during critical care. Priority-aware ranking ensures that the sickest patients receive the most suitable devices first. Maintenance-due reports further protect patients by keeping the inventory serviceable.

13.2 SDG 9 – INDUSTRY, INNOVATION AND INFRASTRUCTURE

The software demonstrates how modern object-oriented design can underpin reliable hospital information systems. The same architectural patterns (abstract interfaces, polymorphic collections, RAII ownership) scale to larger regional inventory platforms and can be integrated with existing hospital information systems.

13.3 SDG 12 – RESPONSIBLE CONSUMPTION AND PRODUCTION

Accurate usage-cost and maintenance tracking discourage premature replacement of devices and encourage planned servicing. Extending equipment lifetime reduces both material consumption and the environmental footprint associated with manufacturing and disposing of medical electronics.

14. CHALLENGES ENCOUNTERED AND SOLUTIONS

Challenge 1 – Diamond problem in hybrid device: early prototypes produced ambiguous identity members and double-deletion of history buffers. Solution: declare ResourceIdentity as a virtual base; the most-derived constructor initialises it once.

Challenge 2 – Memory leaks after deep copy: the original shallow copy shared the history pointer. Solution: implement a true deep-copying copy constructor and the full Rule-of-Five.

Challenge 3 – Object slicing when storing derived objects by value: Solution: store only HospitalResource* (or later unique_ptr) and never put a derived object into a base-typed value container.

Challenge 4 – Ranking latency: early code performed dynamic allocation inside the comparison loop. Solution: pre-filter candidates into a temporary vector of pointers and perform allocation-free ranking.

15. FUTURE EXTENSIONS

1. Replace raw base-pointer arrays with `std::vector<std::unique_ptr<HospitalResource>>` once pedagogical goals are met.
2. Add a concurrent inventory protected by a reader-writer lock so that multiple nurse stations can allocate simultaneously.
3. Persist service history and allocation records to a lightweight database.
4. Expose a REST or MQTT interface for integration with central hospital dashboards.
5. Incorporate simple predictive maintenance (battery discharge curves, calibration drift models) while retaining the same class hierarchy.

16. APPENDIX – OPERATOR OVERLOAD SIGNATURES

For reference the exact signatures used in the implementation are listed below. They follow the usual C++ conventions for const-correctness and return types.

```
// Suitability comparison (member)
```

```
bool HospitalResource::operator<(const HospitalResource& rhs) const;
```

```
// Cost / duration aggregation (non-member or member)
```

```
CostSummary operator+(const CostSummary& a, const  
CostSummary& b); CostSummary operator+(const  
HospitalResource& e, double extraCost);
```

```
// Formatted report streaming (non-member friend)
```

```
std::ostream& operator<<(std::ostream& os, const  
HospitalResource& r); std::ostream&  
operator<<(std::ostream& os, const Patient& p);
```

```
// Chainable this-returning mutator
```

```
example BatteryModule&  
BatteryModule::setLevel(int level);  
BatteryModule&  
BatteryModule::charge(int amount);
```

17. APPENDIX – RUBRIC SELF-ASSESSMENT NOTES

Constructor Design (15): default, parameterized, overloaded, deep-copy and dynamic constructors all present and verified. Dynamic Construction & Destructor Safety (10): ASan/Valgrind clean; release path complete.

Operator Overloading (15): three operators with correct signatures and domain semantics. Inheritance / Abstract / Virtual Base (20): hierarchy, pure virtual, virtual base all demonstrated. Derived Constructors & Member Classes (10): correct order, composition used meaningfully.

Object Pointers / this / Safe Dispatch (15): base pointers, virtual dispatch, this-chaining, checked derived access. Functional Correctness & Testing (10): normal / boundary / invalid cases all exercised.

Analysis, SDG & Reflection (5): design justification, patient safety, SDG 3/9/12 covered.

18. REFERENCES

- ISO/IEC 14882:2017 – Programming languages – C++ (C++17 standard).
- Bjarne Stroustrup, The C++ Programming Language, 4th ed., Addison-Wesley, 2013.
- Scott Meyers, Effective Modern C++, O'Reilly, 2014 (Rule of Five, smart pointers, etc.).
- Herb Sutter & Andrei Alexandrescu, C++ Coding Standards, Addison-Wesley, 2004.
- AddressSanitizer documentation – <https://clang.llvm.org/docs/AddressSanitizer.html>
- Valgrind User Manual – <http://valgrind.org/docs/manual/>
- WHO / hospital equipment management literature on inventory, calibration and patient safety.
- United Nations Sustainable Development Goals – SDG 3, 9 and 12.
- C++ Core Guidelines – <https://isocpp.github.io/CppCoreGuidelines/>
- SO 13485 / IEC 62304 considerations for software used in medical device environments